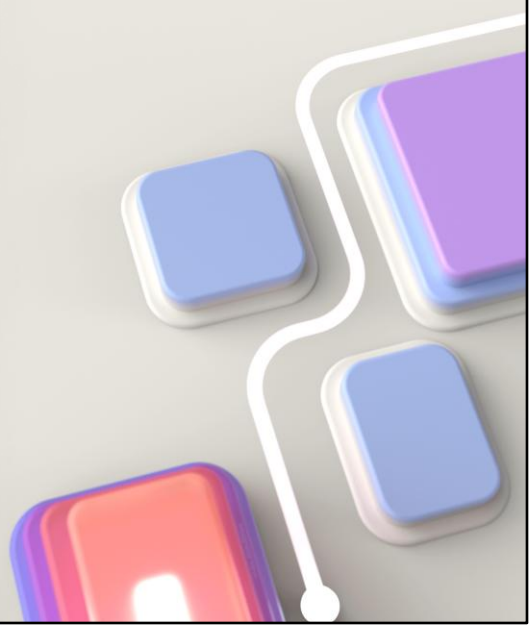
Microsoft

PL-400T00 Microsoft Power Platform Developer

Create a Technical Design



This deck is optional and summarizes course and assists with the Create a Technical Design objectives in the PL-400 exam

This deck effectively summaries the content of the course with the aim of being able to compare the various approaches outlined in the course

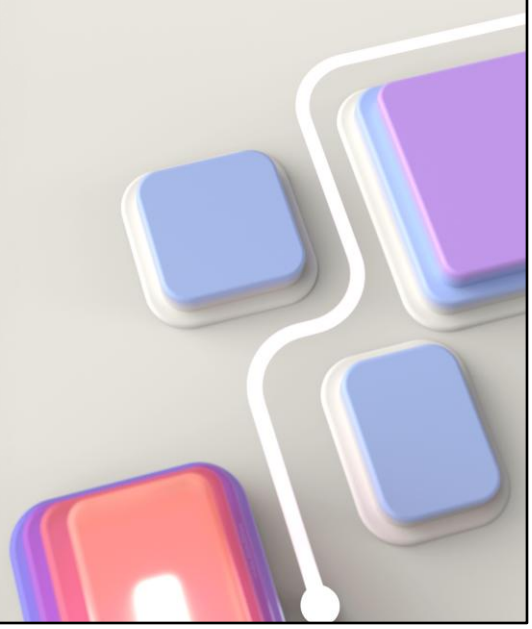
Modules

- Technical architecture

Modules

1. Technical architecture

Module 1: Technical architecture



1 Create a technical design (10–15%)

1.1 Design technical architecture
1.1.1 Design the technical architecture for a solution
1.1.2 Design authentication and authorization strategy
1.1.3 Determine whether requirements can be met with out-of-the-box functionality
1.1.4 Determine when to use Logic Apps versus Power Automate cloud flows
1.1.5 Determine when to use serverless computing, plug-ins, or Power Automate
1.1.6 Determine when to use standard tables, virtual tables, or connectors
1.1.7 Describe security capabilities of the Microsoft Power Platform including data policies (DLP), security roles, teams, business units and row sharing

1.2 Design solution components
1.2.1 Design a Microsoft Dataverse data model
1.2.2 Design Power Apps reusable components including canvas components, code components, and client scripting
1.2.3 Design custom connectors
1.2.4 Design Dataverse code components including plug-ins and Custom APIs
1.2.5 Design automations including Power Automate cloud flows and real-time workflows
1.2.6 Design Azure inbound and outbound integrations

Design authentication and authorization strategy

Authentication is the process or action of verifying the identity of a user or process.

Microsoft Entra ID is used for authentication for accessing the Power Platform.

Users can authenticate using their Entra ID tenant credentials.

When building integrations with Dataverse you may need to register applications in Entra ID and then add S2S accounts to allow external systems to access Dataverse.

When accessing Azure services and external systems you will need to define how to authenticate against those systems. Typically, you will use OAuth 2.0.

Authorization is the process or action of verifying whether an authenticated user is authorized to access the resources that are being provided.

Access to Power Platform environments is controlled by Entra ID security groups.

Access to resources within a Power Platform environment is controlled by a combination of Dataverse security roles and sharing of resources.

Authentication is the process or action of verifying the identity of a user or process. Microsoft's solution to this verification process is Azure Active Directory (Azure AD). Azure AD supports a number of options to verify the identity of a user or process. Abstracting your identity provider allows for a good separation of concerns because managing usernames and passwords can be a difficult (and risky) process.

Authorization is the process or action of verifying whether an *authenticated* user is *authorized* to access the resources that are being provided.

Determine whether requirements can be met with out-of-the-box functionality

As a developer, you should approach apps on Microsoft Power Platform from the perspective that writing code for achieving desired business application functionality should be considered as an exception to no-code and low-code approaches.

You should use the capabilities of the Power Platform instead of writing code whenever possible, for example:

- Rollup columns
- Calculated columns
- Formula columns
- Column-level security

As a developer, you should approach apps on Microsoft Power Platform from the perspective that writing code for achieving desired business application functionality should be considered as an exception to no-code and low-code approaches. However, quality areas such as maintainability, upgradability, stability, and performance should also be considered when you are determining what is the best approach for a given scenario.

Learning what Microsoft Power Platform can do out of the box is important because you do not want to build something that they already do and just needs to be configured. That also applies to the functionality implemented in the available Dynamics 365 apps. For example, invoice calculations using variable multi-currency price lists are complex, but they are well implemented, and time-tested in Dynamics 365 Sales. If this functionality is required, you should consider using Dynamics 365 Sales built-in capabilities instead of implementing your own calculation engine.

You should also recognize that Microsoft Power Platform often implements something in a particular way that benefits the platform. This may be different if you are used to doing it in custom application code. It is not uncommon for developers that are new to building Microsoft Power Platform solutions to try to customize the platform the way they used to build custom apps previously. This should be avoided when possible and you should try to take advantage of what the platform does well, rather than try to change it to what you are used to doing. For example, in the past you may have implemented column-level security using custom code. This is, however, not required in Dataverse where column-level security is an out-of-the-box feature and simply needs to be configured.

Determine whether requirements can be met with out-of-the-box functionality

Calculated columns

- Calculated on retrieve
- Can use columns on table and columns in many-to-one relationships

Formula columns

- Calculated on retrieve
- Can use columns on table and columns in many-to-one relationships
- Supports wide array of Power Fx formulas
- Limited support for Currency columns

Rollup columns

- Requires a one-to-many relationship
- Recalculated every hour
- Max 100 per environment
- Max 10 per table
- Supports hierarchical relationships

Calculated and rollup column limitations

Calculated columns

- Calculated columns can span two tables only.
- A calculated column can contain a column from another table (spanning two tables: current table and parent row).
- A calculated column can't contain a calculated column from another table that also contains another column from a different table (spanning three tables).

Sorting is disabled on:

- A calculated column that contains a column of a parent row.
- A calculated column that contains a logical column (for example, address column).
- A calculated column that contains another calculated column.

Rollup columns

- Rollups calculate a value from a column in one or many child rows in a related table.
- You are limited to a maximum of ten (10) rollups for each parent table.
- Because rollup columns persist in the database, they can be used for filtering or sorting just like regular columns.
- Remember that rollups are asynchronous and are calculated by Dataverse on a set schedule, so you might not see the expected value in a rollup until that job runs.

Formula columns

- Calculated on retrieve
- Can use columns on table and columns in many-to-one relationships
- Supports wide array of Power Fx formulas
- Limited support for Currency columns
- Be aware of timezone behaviors with DateDiff

https://learn.microsoft.com/en-us/power-apps/maker/data-platform/formula-columns

Determine whether requirements can be met with out-of-the-box functionality

1. Calculated, Formula and rollup column values are read-only
2. Calculated, Formula and rollup column values are not audited
3. Calculated, Formula and rollup columns do not trigger workflow, code, or Power Automate cloud flows
4. Calculated columns cannot contain a calculated column from another table that also contains another column from a different table (spanning three tables)

Determine whether requirements can be met with out-of-the-box functionality

Alternatives to calculated and formula columns

- Business rule
- Client scripting (JavaScript)
- Real-time classic workflow
- Dataverse plug-in
- Power Automate cloud flow

Alternatives to rollup columns

- Dataverse plug-in
- Power Automate cloud flow

You may need to create a custom column and calculate the value with plug-in or Power Automate or one of the alternatives

Determine whether requirements can be met with out-of-the-box functionality

Client scripting

Client scripting is actioned immediately in the user interface when the user exits a field

Calculated and formula columns

Calculated columns are calculated on retrieve, so client changes won't show until data is refreshed

Columns are calculated on retrieve, so client changes won't show until data is refreshed. If you want data to update instantly on the form, client scripting or business rules would be the preferred method.

Determine whether requirements can be met with out-of-the-box functionality

When evaluating requirements, you should be fully aware and leverage functional solutions over code-first solutions where possible

Power Automate cloud flows offer and can exceed functionality traditionally available within Dataverse classic workflows and plug-ins for example you can use a Power Automate cloud flow to cascade data changes down a one-to-many relationship e.g., change of address from account to contacts

Business rules can achieve common requirements that negate the need for JavaScript in model-driven app forms

Determine whether requirements can be met with out-of-the-box functionality

JavaScript	Business rules
• Model-driven app form only	• Model-driven app form and Dataverse
• Operates on columns, sections, and tabs	• Operates on columns only
• Use Web API to access any table/column	• Limited to columns on table/form
• Use with command buttons	• Shows error next to field in the form
• Supports complex logic	• Supports simple logic
• Supports OnSave event	• Supports OnLoad and OnChange events only
• Supports Notifications	• Supports Recommendations
• Full Web API support	

Client scripting vs. Business rules

The advantage of business rules is that they are easy to understand and implement for a non-developer, and they can be included as part of a Dataverse solution for deployment in production.

Business rules is an available feature in model-driven applications that enable users to conduct frequently required actions on a form based on certain requirements and conditions.

Business rules can do the following tasks:

- Set column values
- Clear column values
- Set column requirement levels
- Show or hide columns
- Enable or disable columns
- Validate data and show error messages
- Create business recommendations based on business intelligence.

While the available framework is robust, some scenarios might exist where business rules can't fully achieve a given requirement. Luckily in these cases, client scripting can narrow the differences between business rules and a

more custom requirement.

Although supported, JavaScript can risk introducing use of unsupported solutions (e.g. interacting with the DOM)

Determine when to use Logic Apps versus Power Automate cloud flows

Logic Apps

- Logic Apps are licensed on a pay-as-you-go basis (consumption) or as part of as service plan (standard)
- Build in Azure portal or with Visual Studio
- Management, logging, monitoring and alerts, with Azure Monitor and Azure Application Insights

Power Automate cloud flow

- Power Automate is licensed through Power Automate licenses that are either by user or per flow
- Build in Power Automate portal
- Approvals and Notifications supported

https://learn.microsoft.com/en-us/azure/azure-functions/functions-compare-logic-apps-ms-flow-webjobs
https://learn.microsoft.com/en-us/azure/logic-apps/monitor-logic-apps

- Simple to medium complexity
- Non business critical
- Capacity with user license https://learn.microsoft.com/en-us/power-platform/admin/power-automate-licensing/types
- Flow history
- Approvals, Button flows
- Adaptive cards in Microsoft Teams
- Service Principals (for some connectors)

- Continuous integration
- Business critical

- Consumption-based
- Full logging and alerting
- Managed identities

Note: Application Insights is in Preview for Power Automate

Determine when to use serverless computing, plug-ins, or Power Automate

Capability	Power Automate cloud flow	Dataverse classic workflow	Dataverse plug-in
Synchronous or Asynchronous	Asynchronous	Either	Either
Access External Data	Yes, using connectors	No	Yes, using APIs
Maintenance	Makers	Business users	Developers
Can Run As	Current user or flow owner	Current user or workflow owner	Any licensed user, system, or current user
Can Run On Demand	Yes	Yes	No
Can Nest Child Processes	Yes	Yes	Yes
Execution Stage	After	Before/After	Before/After
Triggers	Create, Column Change, Delete, On Demand, Scheduled	Create, Column Change, Delete, On Demand	Create, Column Change, Delete, plus many other messages

Workflows/Power Automate flows vs. plugins

Dataverse offers various methods to implement logic to respond to events in the system, in particular to the data changes such as create, update, delete of the data rows.

Dataverse classic workflow

- Asynchronous or Synchronous
- Actions within Dataverse
- Email templates
- Extend using .NET with custom workflow assemblies
- No additional cost

Dataverse Plug-ins

- Asynchronous or Synchronous
- Actions within Dataverse but can call APIs with come security restrictions
- Integration to Azure Service Bus (IServiceEndpoint)
- .NET assembly
- 2-minute execution time limit

Power Automate cloud flows

- Asynchronous
- Not restricted to Dataverse

Determine when to use serverless computing, plug-ins, or Power Automate

Capability	Dataverse classic workflow	Dataverse plug-in	Client Scripting
Synchronous	Either	Either	Synchronous
Access External Data	No	Yes	Yes (with limitations)
Maintenance	Business Users	Developers	Developers
Can Run As	User	Any licensed user or current user	User
Can Run On Demand	Yes	No	No
Can Nest Child Processes	Yes	Yes	No
Execution Stage	Before/After	Before/After	Before/After
Triggers	Create, Column Change, Status Change, Assign to Owner, On Demand	Create, Column Change, Status Change, Assign to Owner, Delete, along with many other specialized triggers	Column Change or Form Load/Save

Workflows/flows vs. plug-ins/client script

Determine when to use serverless computing, plug-ins, or Power Automate

Azure Functions

- Consumption plan 5-minute default and can increased to 10-minute
- Durable functions allow chaining of functions
- Premium and Dedicated plans have longer
- Need to Authenticate

Dataverse Plug-in

- Event driven
- Cannot schedule
- 2-minute limit
- Authenticated automatically

Azure Functions vs. Dataverse Plug-ins

Microsoft Azure Functions provides a great mechanism for doing small units of work, similar to what you would use plug-ins for in Dataverse. In many scenarios it might make sense to offload this logic into a separate component, such as an Azure Function, to reduce load on the Dataverse's application host. You have the availability to run functions in a synchronous capacity because Dataverse webhooks provide the Remote Execution Context of the given request.

However, Azure Functions does not explicitly run within the Dataverse's implementation pipeline, so if you need to update data in the most high-performing manner, such as autoformatting a string value before it posts to Dataverse, we still recommend that you use a plug-in to perform this type of operation.

Azure Functions

- 5 or 10 minute execution time limit
- Call with webhook
- Can be consumed by Custom Connector
- Requires Azure subscription

Design Azure inbound and outbound integrations

Service Bus

- Asynchronous
- Register with Plugin Registration Tool
- Invoke with no code or with code
- Requests can be queued

Webhook

- Synchronous or Asynchronous
- Register with Plugin Registration Tool
- Invoke with no code or with code
- Requests dependent on HTTP endpoint being available

Webhooks vs. Azure Service Bus

When considering integration mechanisms, you have a few available options. It's important that you consider various elements when choosing a given method.

Consider using Azure Service Bus when:

- High scale asynchronous processing/queueing is a requirement.
- Multiple subscribers might be needed to consume a given Dataverse event.
- You want to govern your integration architecture in a centralized location.

Consider using webhooks when:

- Synchronous processing against an external system is required as part of your process (Dataverse only supports asynchronous processing against Service Bus Endpoints).
- The external operation that you are performing needs to occur immediately.
- You want the entire transaction to fail unless the webhook payload is successfully processed by the external service.
- A third-party Web API endpoint already exists that you want to use for integration purposes.
- SAS authentication isn't preferred and/or feasible (webhooks support authentication through authentication headers and query string parameter keys).

https://learn.microsoft.com/power-apps/developer/data-platform/use-webhooks

Design multi-lingual and accessible code components

- Define and use RESX files and use them in multiple areas:
  - PCF controls
  - JavaScript
    - Xrm.Utility.getResourceString("new_/strings/MyAppResources","hello")
  - Model-driven app / Dataverse interface
- General accessibility guidelines:
  - Add tooltips and accessible text definitions
  - Consider tab order settings
  - Be aware of color standards and ratios
  - Always include appropriate text labels
  - Align to industry standards
    - WCAG
- Use Fluent UI where appropriate

What are RESX files?

RESX web resources contain the keys and localized string values for a single language defined using the RESX XML format. RESX is a common format for defining localized resources for windows applications, so there is common tooling available to work with this type of file and localization vendors will be familiar with working with them. When the file is published as a web resource in CRM it will be converted to a JSON format which will be downloaded to the application when needed.

The key benefit of using them is reusability – defined once, the RESX file can be used across various customized and developed assets that we build.

Optional instructor demo – Demonstrate the following the sample PCF control, that makes use of RESX files:

https://learn.microsoft.com/en-us/power-apps/developer/component-framework/sample-controls/localization-api-control

What is Fluent UI?

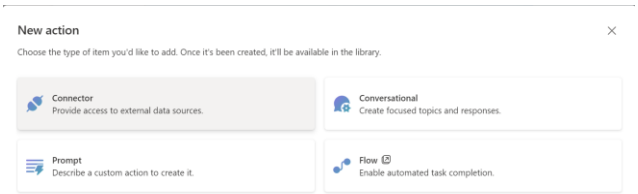
Fluent is an open-source, cross-platform design system that gives designers and developers the frameworks they need to create engaging product experiences—accessibility, internationalization, and performance included.

The benefits of using Fluent UI, particularly from a PCF control standpoint, is that not only will the controls you build align nicely with the default Power Apps experience, you will also be able to tap into numerous capabilities that can aid from an accessibility standpoint – such as tooltips and inheriting default color themes.

Extending copilots with Copilot Studio

Copilot Studio **offers several ways to design your plugins** to suit your specific needs. Plugins are used both to extend Microsoft Copilot, and with a custom copilot. They can **call services, perform actions, and provide answers.**

Power Platform connector	A connector is a wrapper around an API that allows the underlying service to talk to Copilot Studio. It provides a way for users to connect accounts and leverage a set of prebuilt actions and triggers to build their apps and workflows. Custom connectors let your plugin retrieve and update data from external sources accessed through APIs.
Power Automate flows	Flows can be called from within a Microsoft Copilot chat that can perform actions or retrieve information across the end user's environment.
Conversational AI plugins	The ability to author a topic-like experience using a visual workflow design. This includes 1200+ power platform connectors, generative answers, Power Automate flows and more.
AI builder prompts	Enable your users to use plain language to get answers and perform actions with Microsoft Copilot. They use natural language understanding (NLU) to understand a user's intent and map it to an associated piece of information, data, or activity.



As part of technical designs, you should consider where extending copilots is the appropriate implementation option. This provides an overview of the options for extending from Copilot Studio.

https://learn.microsoft.com/en-us/microsoft-copilot-studio/copilot-plugins-overview

Exercise: Evaluate configure vs code options

Review three scenarios and determine how meet them with an out-of-the-box, no-code, low-code, or code-first approach.

- **Evaluate scenarios**
- **Discuss pros and cons of options**

Exercise: Scenario 1

When a record is created, child records must be created automatically.

Assume that the parent record is a sports season, and the child records would be games to be played in that season. For this challenge, you'll always want to create 12 games for each season.

Season: Summer 2023

Games: Game 1, Game 2, Game 3, and so on

- **In which ways could you accomplish this?**
- **What more information do you require to decide the most appropriate approach?**

This task could be accomplished through either Power Automate or a Dataverse plug-in or even an Azure Function via a webhook.

- With Power Automate, you would define and increment a variable for the name of each game record. The records would be created asynchronously.
- With a plug-in, you would also define a variable in the code and create each record. The records could be created either synchronously or asynchronously.

The requirement of accomplishing the task through Power Automate or a plug-in would be decided based on when you need the records created (immediately or as soon as possible) and the available resources needed to implement the requirement.

In addition, if the requirement is to block the creation of the record, if for any reason any child records were unable to be created, a synchronous plug-in would be required.

Exercise: Scenario 2

The system has a choice column with 17 values.

You need to filter out invalid values on the form in a model-driven app.

In this global choice, you might have values that apply across multiple tables.

A small number of them could only apply on a subset of all the tables that you might use with this choice.

Another possibility is that only certain values from the 17 would apply based on other available values such as a status field.

How could you accomplish this?

With the values being a subset of the choice, writing custom code with JavaScript is the only way to solve this requirement.

Exercise: Scenario 3

You have an option set with three options; Gold, Silver, and Bronze.

When the user selects Gold or Silver, you need to set a field on the form as required.

When the user selects Bronze, you need to display a notification on the form.

This can be analyzed into two sets of requirements:

- Requirement 1: When the user selects Gold or Silver, you need to set a field on the form as required.
- Requirement 2: When the user selects Bronze, you need to display a notification on the form.

How could you accomplish each of these requirements?

A business rule would work well in this situation because it would show and hide fields based on these criteria

- Requirement 1 can be accomplished quickly by using a business rule or form client scripting.
- Requirement 2 cannot be done by using a business rule because they do not support form notifications; it must be done by using client scripting. in this situation because it would show and hide fields based on these criteria. However, with the values being from a choice, writing client script code is the only way to solve this requirement.

Summary

- The Power Platform offers many different ways to meet requirements with out-of-the-box features, no-code and low-code apps and flows, and code-first options.
- As a developer, you should approach the Microsoft Power Platform from the perspective that writing code for achieving desired business application functionality should be considered as an exception to no-code and low-code approaches.
- Qualities such as maintainability, upgradability, stability, and performance should also be considered when you are determining what is the best approach for a given scenario.

Modules

1. Solutions and Application Lifecycle Management

Microsoft